

千葉大学大学院融合理工学府博士前期課程
2021 年 10 月入学及び 2022 年 4 月入学
入学試験問題
【数学情報科学専攻情報科学コース】
専門科目

Specialized Subject

Entrance Examination Questions for
Master's Program of October Admission 2021/April Admission 2022 to
the Department of Applied and Cognitive Informatics,
the Division of Mathematics and Informatics,
Graduate School of Science and Engineering, Chiba University

注意事項

- (1) 「解答はじめ」の合図があるまで、この問題冊子を開いてはいけません。
- (2) 試験開始後の退室は認めません。ただし、トイレ等のため退室したい場合は申し出てください。
- (3) すべての解答用紙に受験番号を記入しなさい。
- (4) すべての問題を解答すること。

Instructions

- (1) You are not allowed to open this question booklet until the proctor has announced "*Kaito hajime.*" in Japanese.
*"*Kaito hajime.*" means "Begin."
- (2) You are not allowed to leave the examination room after the exam has started. However, if you need to use the restroom during the test, raise your hand, and wait for permission to leave the room.
- (3) Fill in your Candidate number at the top of each answer sheet.
- (4) Answer all questions.

問 1

U, V がともに空でない有限集合 (finite set) とし, それぞれの要素 (element) の数を $m, n \in \mathbb{N}$ ($\mathbb{N}$ は, 自然数 (natural number) の集合) とする。 U と V の要素の間に何らかの対応が定まっていて, かつ, U の 1 つの要素に対応する V の要素は高々 1 つしかないという対応関係 (correspondence) を考える。以下の各問いに答えなさい。

- (1) U のすべての要素について, 対応する V の要素が存在する場合の対応関係を U から V への写像 (mapping) という。 U から V への相異なる写像はいくつあるか, m, n を用いて答えなさい。
- (2) U の要素によっては, 対応する V の要素が存在しないこともある場合の対応関係を U から V への部分写像という。 U から V への相異なる部分写像 (partial mapping) はいくつあるか, m, n を用いて答えなさい。
- (3) 任意の $u_1, u_2 \in U$ について, 次を満たすとき, 写像 f を U から V への単射 (injective mapping, one-to-one mapping) という。

$$u_1 \neq u_2 \text{ ならば, } f(u_1) \neq f(u_2)$$

$m \leq n$ のとき, U から V への相異なる単射はいくつあるか, m, n を用いて答えなさい。

- (4) すべての $v \in V$ について, それぞれ, ある $u \in U$ が存在して, $f(u) = v$ が成り立つとき, 写像 f を U から V への全射 (surjection, onto mapping) という。 $m = 4, n = 3$ のとき, U から V への相異なる全射はいくつあるか答えなさい。

問 2

n を 3 以上の自然数として、参加者が $A_1, A_2, \dots, A_n$ の n 人であるパーティを考える。どの参加者 2 人についても、それぞれ互いに面識があるかないかのどちらかであり、どの参加者同士も互いの関係の認識に相違はないものとする。例えば、 A_1 さんが A_2 さんと面識があると認識している場合には、 A_2 さんも A_1 さんと面識があると認識している。各参加者 A_i がパーティ参加者内の何人と面識があるかを $F(A_i)$ で表す。例えば、 A_1 さんが最近知り合った A_2 さんに、 A_2 さんのことを知らない旧友 A_3 さんを紹介するため、3 人で小さなパーティを開く場合、 A_1 さんは残り 2 人の両方と面識があるので $F(A_1) = 2$ であるが、逆に A_2 さんや A_3 さんは A_1 さん以外の参加者とは面識がないため、 $F(A_2) = F(A_3) = 1$ となる。

さて、 n 人でパーティを開いた場合において、どの 2 人も互いに面識がないような参加者の一群に着目する。例えば、新入社員の A_1 さんが会社の先輩 A_2 さん、10 年ぶりに会うことになる旧友 A_3 さん、大学からの親友 A_4 さん、同じく大学からの親友で、 A_4 さんとの共通の友人 A_5 さんの 4 人を自宅に呼ぶとする。この場合、 A_2, A_3, A_5 の 3 人については、誰もお互いに面識はないであろう。実は一般に、どのようなパーティであれ参加者が n 人の場合、少なくとも

$$\sum_{i=1}^n \frac{1}{F(A_i) + 1}$$

人以上からなる、どの 2 人も互いに面識がない参加者の一群がいることが古くから知られている。この事実を証明するため、以下の各問いに答えなさい。(3) は、日本語または英語で答えなさい。

- (1) n 人の参加者 $A_1, A_2, \dots, A_n$ に、各自の来場順を示す 1 から n の数字が書かれた札を 1 枚ずつ渡すことを考える。各参加者が来場する順序は無作為に決まるとし、同時に 2 人以上が到着することはない、 $n!$ 通りあるどの番号札の行き渡り方も等しく確率 (probability) $\frac{1}{n!}$ で実現されるものとする。さて、参加者が全員揃ったところで、各参加者に自分と面識のある者全員と番号札を見せ合ってもらい、自分が最も遅く来場していた場合、つまり面識のある参加者は皆自分より若い番号の札を持っていた場合には挙手してもらい、そうでない場合には手を下げていてもらうとする。各参加者は正直にこの指示に従うと仮定する。参加者 A_i が挙手する確率を、その参加者と面識のある参加者の数 $F(A_i)$ を用いて表しなさい。
- (2) n 人の参加者のうち、挙手する参加者の総数について、その期待値 (expected value) を、 $F(A_1), F(A_2), \dots, F(A_n)$ を用いて表しなさい。
- (3) n 人の参加者の中に、どの 2 人も互いに面識がない

$$\sum_{i=1}^n \frac{1}{F(A_i) + 1}$$

人以上からなる参加者の一群がいることを証明しなさい。

問 1

赤色、青色、緑色の3色のLEDがある。このLEDは、2つの入力信号 z_1z_0 が00ならば赤、01ならば青、10ならば緑、11ならばすべて点灯するというものである。2進数の同期型順序回路 (binary synchronous sequential circuit) によって、クロックが立ち上がるたびにLEDの色が変化するような回路を設計したい。以下の各問いに答えなさい。ただし、ここで用いて良い論理回路素子 (logical circuit element) は、AND, OR, NOT, NAND, D フリップフロップ (D flip-flop) とする。また、図 2-1 は、クロックが立ち上がるたびに z_1z_0 の状態が変化する同期型順序回路を汎用的に描いたものである。

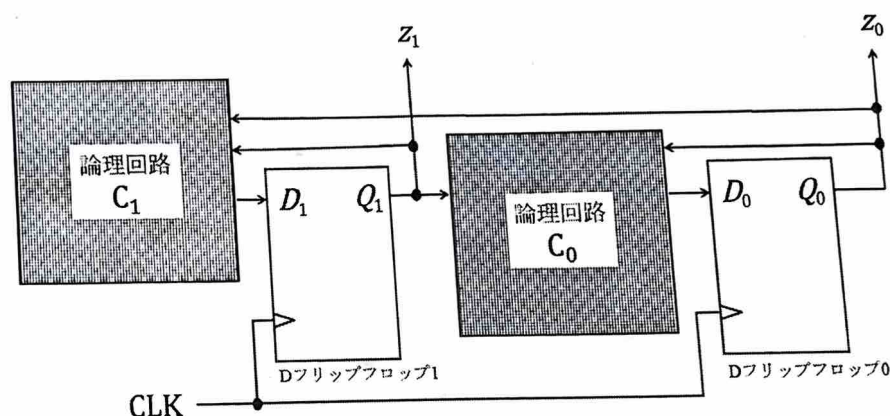


図 2-1 同期型順序回路 1

- (1) ポジティブエッジトリガー型 D フリップフロップ (positive-edge-triggered D-flip-flop) の特性表 (characteristic table) の空欄を埋めなさい。ただし、表 2-1 において、CLK はクロック入力、D は D フリップフロップの入力信号、Q は出力信号を表す。

表 2-1 特性表 (characteristic table)

CLK	D	Q
0 → 1	0	
	1	
上記以外	0	
	1	

$Q_1 Q_0$	$D_1 D_0$
00	01
01	10
10	00

$Q_1 Q_0$	$D_1 D_0$
00	01
01	10
10	00

$Q_1 Q_0$	$D_1 D_0$
00	01
01	10
10	11
11	00

- (2) z_1z_0 が, 00, 01, 10, 11, 00, 01, ...の順に状態変化する2進数の同期型順序回路(同期式4進カウンタ, synchronous quaternary counter)を設計する。図2-1の C_1, C_0 に相当する各論理式(logical formula) D_1, D_0 のそれぞれを, Q_1, Q_0 を用いて表しなさい。
- (3) z_1z_0 が, 00, 01, 10, 00, 01, ...の順に状態変化する2進数の同期型順序回路(同期式3進カウンタ, synchronous ternary counter)を設計する。図2-1の C_1, C_0 に相当する各論理式 D_1, D_0 のそれぞれを, Q_1, Q_0 を用いて表しなさい。
- (4) z_1z_0 が, 10, 01, 00, 10, 01, ...の順に状態変化する2進数の同期型順序回路(同期式3進ダウンカウンタ, synchronous ternary down counter)を設計する。図2-1の C_1, C_0 に相当する各論理式 D_1, D_0 のそれぞれを, Q_1, Q_0 を用いて表しなさい。
- (5) 制御信号 S が0ならば, 赤, 青, 緑の順に, S が1ならばその逆順に点灯するシステムを, 2進数の同期型順序回路によって設計したい。制御信号 S が0ならば, z_1z_0 が, 00, 01, 10, 00, 01, ...の順に, S が1ならば, z_1z_0 が, 00, 10, 01, 00, 10, ...の順に状態変化する2進数の同期型順序回路を設計する。これは, 図2-2に示すように, S も入力の一部とした2つのDフリップフロップを用いて構成できる。図2-2の C'_1, C'_0 に相当する各論理式 D_1, D_0 のそれぞれを, Q_1, Q_0, S を用いて表しなさい。

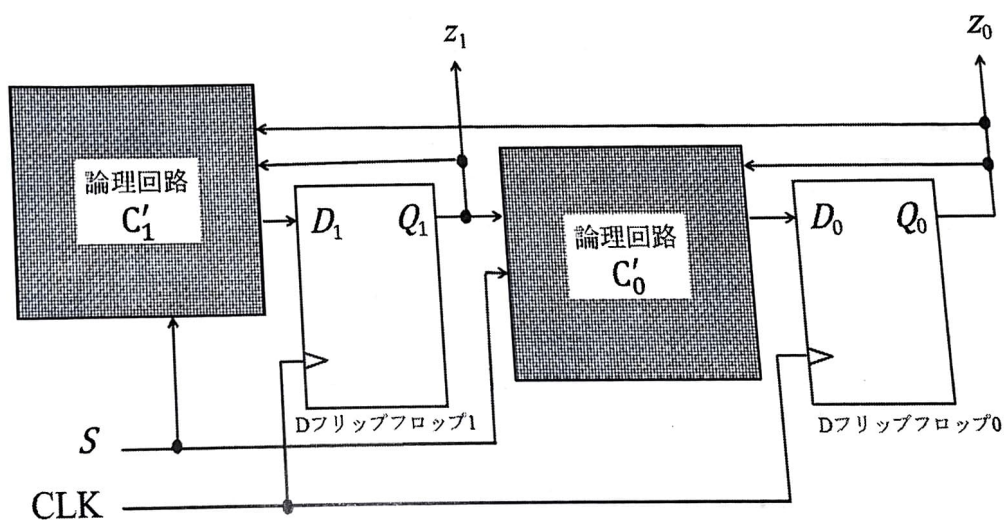


図2-2 同期型順序回路2

S	Q ₁	Q ₀	D ₁	D ₀
0	0	0	0	1
0	0	1	1	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	1
1	1	1	1	1

Q ₁ Q ₀	00	01	11	10
S=0	1	1	1	1
S=1	1	1	1	1

$$S \cdot \bar{Q}_1 \cdot \bar{Q}_0 + S \cdot Q_0$$

Q ₁ Q ₀	00	01	11	10
S=0	1	1	1	1
S=1	1	1	1	1

$$S \cdot \bar{Q}_1 \cdot \bar{Q}_0 + S \cdot Q_0$$

問 2

Internet Protocol Version 4 (IPv4) を用いるコンピュータネットワーク (computer network) について、以下の各問いに答えなさい。ただし、(3) および (4) は、日本語または英語で答えなさい。

- (1) あるルータ (router) において、表 2-2 で示される経路表に従ってパケット (packet) を転送する。このとき、以下のパケットはそれぞれの IP アドレス (IP address) に転送されるかを答えなさい。

パケット A 宛先: 192.168.20.5

パケット B 宛先: 192.168.20.240

パケット C 宛先: 192.168.20.70

表 2-2 経路表 (routing table)

Destination	Next Hop	Subnet Mask
192.168.20.0	192.168.21.250	255.255.255.192
192.168.20.64	192.168.21.251	255.255.255.192
192.168.20.128	192.168.21.252	255.255.255.192
192.168.20.192	192.168.21.253	255.255.255.192
0.0.0.0	192.168.21.254	0.0.0.0

- (2) 図 2-3 のようなネットワークにおいて、HOST 1 から traceroute 192.168.6.1 を実行した時に表示される IP アドレスを、表示される順に答えなさい。ただし、HOST 1 および HOST 2 は UNIX 系オペレーティングシステム (Operating System) が搭載されており、traceroute コマンドが使用できる状態に設定している。
- (3) IPv4 では、IP パケットヘッダ (IP packet header) に TTL (Time-to-Live) フィールドが設けられている。TTL 値とは何かを、ルータという用語を用いて説明しなさい。また、IPv4 パケットフォーマットに含まれる TTL フィールドの役割を説明しなさい。
- (4) (3) を踏まえて、traceroute がどのように経路を探索するかを、TTL という用語を用いて説明しなさい。

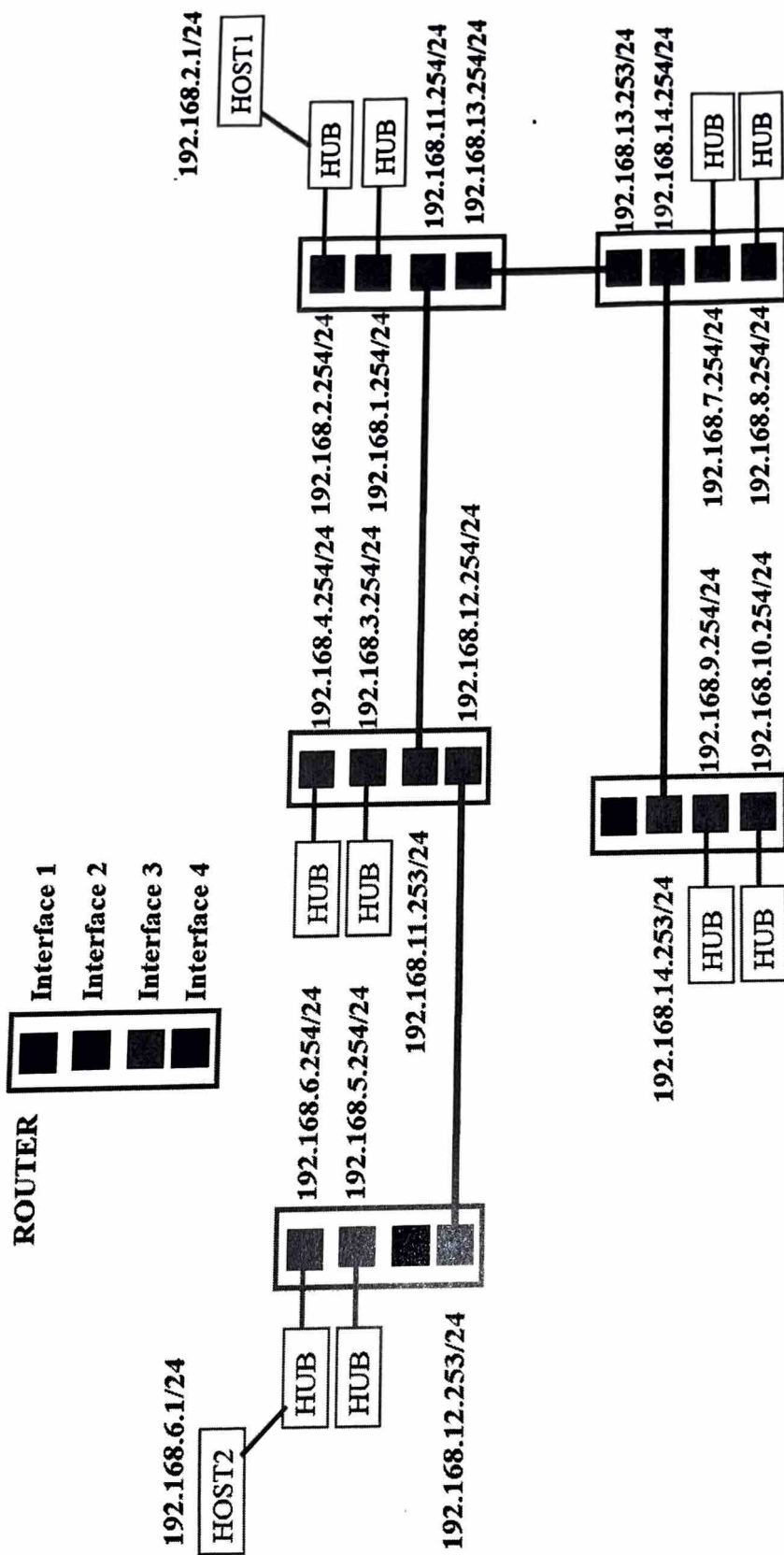


図 2-3 ネットワーク構成図

問 3

非決定性有限オートマトン (nondeterministic finite automata) は、現在の状態 (state) と入力に対する状態遷移 (state transition) が一意に定まらないことがある有限オートマトンである。非決定性有限オートマトンでは、入力記号列を最後まで読み込んだ時に終了状態 (final state) にいるような状態遷移列が 1 個以上ある場合に、その入力を受け (accept) したという。図 2-4 は入力記号が $\{0, 1\}$ 、状態集合が $\{P_0, P_1, P_2\}$ 、開始状態が P_0 、終了状態の集合が $\{P_2\}$ の非決定性有限オートマトンの状態遷移図 (state transition diagram) の例である。以下の各問いに答えなさい。

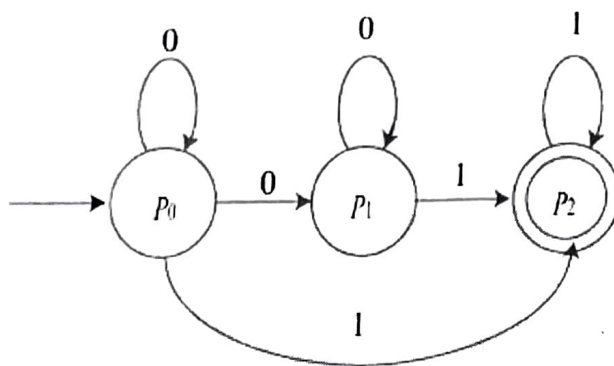


図 2-4 非決定性有限オートマトンの状態遷移図の例

- (1) 図 2-4 のオートマトンに記号列 000111 を入力して受理された時の状態遷移列を示しなさい。入力記号を最後まで読んだときに終了状態にいる状態遷移列を 1 個示せばよい。例えば、001 が受理される状態遷移列の一例は $P_0P_1P_1P_2$ と表される。
- (2) 以下の表 2-3 は上の状態遷移図を表にしたものである。遷移先がない場合は遷移先を空集合 ($\emptyset$, empty set) と書いている。解答用紙に書かれた表の空欄を埋めなさい。

表 2-3 図 2-4 のオートマトンの状態遷移表

状態 \ 入力	0	1
P_0	P_0, P_1	
P_1		P_2
P_2	$\emptyset$	

図 2-4 の非決定性有限オートマトンと等価な決定性有限オートマトン (deterministic automaton) を構成することを考える。その際、非決定性有限オートマトンの状態集合の部分集合 (subset) を、構成する決定性有限オートマトンの状態とする。例えば、状態 P_0 で 0 が入力された時の遷移先は P_0 または P_1 であるが、決定性有限オートマトンでは状態 $[P_0]$ で 0 が入力された時の遷移先を状態 $[P_0, P_1]$ とする。

- (3) 状態集合 $\{P_0, P_1, P_2\}$ の部分集合をすべて書きなさい。
- (4) 以下の表 2-4 は、表 2-3 の非決定性有限オートマトンと等価な決定性有限オートマトンの状態遷移表である。解答用紙に書かれた表の空欄を埋めなさい。

表 2-4 表 2-3 と等価な決定性有限オートマトンの状態遷移表

入力 \ 状態	0	1
$[P_0]$	$[P_0, P_1]$	
$[P_2]$	$\emptyset$	$[P_2]$
$[P_0, P_1]$		

- (5) 表 2-4 のオートマトンは 3 状態を持つが、そのうち 2 状態を 1 状態にまとめることができる。これにより状態数が 2 である、表 2-4 と等価なオートマトンを構成できる。このオートマトンの状態を P_A, P_B とする。以下の表 2-5 はそのオートマトンの状態遷移表である。解答用紙に書かれた表の空欄を埋めなさい。

表 2-5 状態数を削減した表 2-4 と等価な決定性有限オートマトンの状態遷移表

入力 \ 状態	0	1
P_A		
P_B	$\emptyset$	

3

次の構造体 (structure) を用いた C 言語 (C language) によるプログラムについて、以下の各問いに答えなさい。

```
typedef struct N{  
    int val;  
    struct N* lt;  
    struct N* rt;  
}node;
```

問1 上記の構造体を用いて次のように配列 (array) n を定義したとき、配列 n のようなデータ構造を一般に何と呼ぶか答えなさい。

```
node n[5];  
  
n[0].val = 0;  
n[0].lt = &n[1];  
n[0].rt = &n[4];  
n[1].val = 1;  
n[1].lt = &n[2];  
n[1].rt = &n[3];  
n[2].val = 2;  
n[2].lt = NULL;  
n[2].rt = NULL;  
n[3].val = 3;  
n[3].lt = NULL;  
n[3].rt = NULL;  
n[4].val = 4;  
n[4].lt = NULL;  
n[4].rt = NULL;
```

- 問2 次のコードは、問1で示した配列 `n` のような構造を持つデータがあるとき、その要素を走査(`traverse`)し、全ノードの `val` の値を重複なく列挙表示する関数 `traverse` を表す。空欄を埋めてコードを完成させなさい。

```
void traverse(node* n) {  
    if (  )  
        return;  
  
    printf("%d ", n->val);  
      
      
  
    return;  
}
```

- 問3 問1で示した配列 n のような構造を持つデータが2つあるとする。それらデータの構造が val の値まで含めて完全に一致 (identical) するかを調べたい。次のコードは、深さ優先で調べる (depth-first search) ことで、一致する場合には0を、それ以外の場合には1を返す関数 `dfs_match` を表す。空欄を埋めてコードを完成させなさい。

```
void dfs_match(node* n1, node* n2) {  
    if (n1 == NULL) {  
        if (n2 != NULL) {  
            ㉔  
        }  
        ㉕  
    }  
    if (n2 == NULL)  
        ㉖  
  
    if ( ㉗ )  
        return 1;  
    if ( ㉘ )  
        return 1;  
    if ( ㉙ )  
        return 1;  
  
    return 0;  
}
```


問4 次のコードは、関数 `dfs_match` を幅優先で調べる (breadth-first search) ように書き換えた関数 `bfs_match` を表す。ただし、コード中の定数 `PLENTY` は比較を行うデータに対し十分なサイズであるものとする。空欄を埋めてコードを完成させなさい。

```
void bfs_match(node* n1, node* n2) {
```

```
    node* np1[PLENTY];
```

```
    node* np2[PLENTY];
```

```
    int head = 0;
```

```
    int tail = 0;
```

```
    [ J ]
```

```
    [ K ]
```

```
    while ( [ L ] ) {
```

```
        [ M ]
```

```
        [ N ]
```

```
        if (n1 != NULL && n2 == NULL)
```

```
            [ O ]
```

```
        if (n1 == NULL && n2 != NULL)
```

```
            [ P ]
```

```
        if (n1 != NULL && n2 != NULL) {
```

```
            if ( [ Q ] ) {
```

```
                [ R ]
```

```
            } else {
```

```
                [ S ]
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```